

Indian Institute of Technology, Hyderabad
Department of Computer Science and Engineering

ID1063: Introduction to Programming
Mid-Semester Examination(Set-A)

Roll Number: _____

Duration: 135 minutes

Total Marks: 100

Lecture Hall Venue: _____

Date: 08.10.2025

Section-Wise Marking Scheme

Section	Type of question	Number of Questions	Marks	Marks Obtained
Section 1	Multiple Choice Questions (MCQ)	10	20	
Section 2	Multiple Select Questions (MSQ)	4	9	
Section 3	Numerical Answer Type (NAT)	5	15	
Section 4	Long Answer Questions	4	26	
Section 5	Coding Questions	5	30	

INSTRUCTIONS PAGE

1. Please fill your roll number correctly and legibly in the space above. Write your full roll number including department code etc.
2. Do NOT turn over this page till the start of exam is announced (6:45pm). Once exam starts you may turn over and read the questions overleaf.
3. Please ensure you are carrying only a pen/pencil and your ID card with you.
4. Do NOT carry any electronic devices like mobiles, watches etc. In case invigilators find any device with you, or any loose sheets/notes, you will be awarded FR, even if you were not intending to use it.
5. Please write your answers in this question paper itself. Please write legibly and within the space provided. Submit the filled in question paper at end of exam to the invigilator (9pm).
6. In the unlikely situation where you don't have space in the question paper, you may write on a blank sheet and submit it. Do not forget to write your roll number on such a page. Staple it along with this question paper.
7. You may use blank sheets given to you for rough work, which should also be submitted back to us. Staple it along with this question paper. Do not forget to write your roll number on such a page. Unidentified rough sheets found near you may make you a suspect for copying! Please be careful.
8. Any attempt to copy/cheat etc. will straight away lead to an FR grade in the course.
9. After finishing your exam at 9pm, stop writing. Do not write anything beyond 9pm. Leave your filled question paper as it is on your desk and move away immediately towards the big screen. The invigilators will collect your sheets, do a tally. After they announce, you can leave the exam hall.

1 Multiple Choice Questions

Note:- Only one of the given options is correct. Circle the correct option among the given options. No partial marking will be given.

[20 points]

1. (2 points) What will be the output of the following code?

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main() {
5     char str[] = "abcCBA";
6     int n = strlen(str); //strlen defines string length
7     int count = 0;
8
9     for(int i=0; i<n; i++) {
10         if(str[i] >= 'a' && str[i] <= 'z')
11             str[i] = str[i] - 'a' + 'A';
12         else if(str[i] >= 'A' && str[i] <= 'Z')
13             str[i] = (str[i] - 'A' + 1) % 26 + 'A';
14     }
15
16     for(int i=0; i<n; i++) {
17         if(str[i] == str[n-1-i]) count++;
18     }
19
20     printf("%s %d", str, count);
21     return 0;
22 }
```

A. ABCDCB 0 B. ABCDBC 2 C. BCDDBC 3 D. BCDDCB 6 E. BCDDCB 3 F. None of the given options

A

2. (2 points) What will be the output of the following code?

```
1 #include <stdio.h>
2 void fun(int arr[]) {
3     arr[0] = 100;
4 }
5 int main() {
6     int arr[3] = {1, 2, 3};
7     fun(arr);
8     printf("%d", arr[0]);
9     return 0;
10 }
```

A. 1 B. 100 C. 2 D. 3 E. Compiler Error F. None of the given options

B

3. (2 points) What will be the output of the following code?

```
1 #include <stdio.h>
2 int main() {
3     int x=0, y=5;
4     while(x=y--) {
5         printf("%d ", x);
6     }
7     return 0;
8 }
```

A. 5 4 3 2 1 B. 4 3 2 1 0 C. 5 D. Infinite Loop E. Garbage Value F. None of the given options

A

4. (2 points) What will be the output of the following code?

```
1 #include <stdio.h>
2 int main() {
3     int i;
4     for(i=0; i<5; i++);
5     printf("%d", i);
6 }
```

```

6     return 0;
7 }

```

- A. 0 1 2 3 4 B. 1 2 3 4 5 C. 0 D. 5 E. Compilation Error F. None of the given options

D

5. (2 points) What will be the output of the following code?

```

1 #include <stdio.h>
2 int main() {
3     int x=0, y=1;
4     if(x)
5         if(y) printf("A");
6     else
7         printf("B");
8     return 0;
9 }

```

- A. A B. B C. Doesn't print anything D. Compilation Error E. xy F. None of the given options

C

6. (2 points) What will be the output of the following code?

```

1 #include <stdio.h>
2 int main() {
3     int x=2;
4     switch(x) {
5         case 1: x+=1;
6         case 2: x+=2;
7         case 3: x+=3;
8         default: x+=5;
9     }
10    printf("%d", x);
11    return 0;
12 }

```

- A. 2 B. 7 C. 10 D. 12 E. 11 F. None of the given options

D

7. (2 points) What will be the output of the following code?

```

1 #include <stdio.h>
2 void fun(int a[3][3], int n) {
3     int sum = 0;
4     for(int i=0; i<n; i++)
5         for(int j=0; j<n; j++)
6             if(i+j == n-1) sum += a[i][j];
7     printf("%d", sum);
8 }
9 int main() {
10    int arr[3][3] = {{1,2,3},{4,5,6},{7,8,9}};
11    fun(arr,3);
12    return 0;
13 }

```

- A. 15 B. 17 C. 18 D. 20 E. 22 F. None of the given options

A

8. (2 points) What will be the output of the following code?

```

1 #include <stdio.h>
2 void modify(int a[2][2]) {
3     a[0][0] += a[1][1];
4     a[1][0] -= a[0][1];
5 }
6 int main() {
7     int arr[2][2] = {{1,2},{3,4}};
8     modify(arr);
9     printf("%d %d %d %d", arr[0][0], arr[0][1], arr[1][0], arr[1][1]);
10    return 0;
11 }

```

- A. 1 2 3 4 B. 5 2 1 4 C. 5 2 3 4 D. 1 2 1 4 E. 1 5 1 4 F. None of the given options

B

9. (2 points) What will be the output of the following code?

```
1 #include <stdio.h>
2 int recur(int a[5], int n) {
3     if(n==0) return 0;
4     a[n-1] += 1;
5     return a[n-1] + recur(a, n-1);
6 }
7 int main() {
8     int arr[5] = {1,2,3,4,5};
9     int x = recur(arr,5);
10    printf("%d %d", x, arr[0]);
11    return 0;
12 }
```

A. 20 1 B. 20 2 C. 25 1 D. 25 2 E. 20 4 F. None of the given options

B

10. (2 points) What will be the output of the following code?

```
1 #include <stdio.h>
2 int fun(int n) {
3     int s=0;
4     while(n>0) {
5         switch(n%3) {
6             case 0: s += n; break;
7             case 1: s -= n; break;
8             case 2: s += n*n; break;
9         }
10        n--;
11    }
12    return s;
13 }
14 int main() {
15    printf("%d", fun(4));
16    return 0;
17 }
```

A. 10 B. 12 C. 14 D. 16 E. 2 F. None of the given options

E

2 Multiple Select Questions

Note:- One or more than one options may be correct. Circle all the correct option(s) among the given options. Partial marking is given only when all the answered options are correct.

[9 points]

1. (2 points) The following four functions perform multiplication of a 2×2 matrix with itself. Assume a correct `main()` function reads a 2×2 matrix and passes it to these functions in accordance with these parameters. Examine the functions carefully. Choose all options that correctly describe the outputs of these functions for any valid input matrix.

```
1 // Snippet 1
2 void func1(int A[2][2], int B[2][2], int C[2][2]) {
3     int i,j,k;
4     for(i=0;i<2;i++)
5         for(j=0;j<2;j++) {
6             C[i][j]=0;
7             for(k=0;k<2;k++)
8                 C[i][j]+=A[i][k]*B[k][j];
9         }
10 }
```

```
1 // Snippet 2
2 void func2(int A[2][2], int B[2][2], int C[2][2]) {
3     int i,j;
4     for(i=0;i<2;i++)
5         for(j=0;j<2;j++)
6             C[i][j]=A[i][0]*B[0][j] + A[i][1]*B[1][j];
7 }
```

```

1 // Snippet 3
2 void func3(int A[2][2], int B[2][2], int C[2][2]) {
3     int a=A[0][0], b=A[0][1], c=A[1][0], d=A[1][1];
4     int p=B[0][0], q=B[0][1], r=B[1][0], s=B[1][1];
5     int M1=(a+d)*(p+s), M2=(c+d)*p, M3=a*(q-s), M4=d*(r-p);
6     int M5=(a+b)*s, M6=(c-a)*(p+q), M7=(b-d)*(r+s);
7     C[0][0]=M1+M4-M5+M7;
8     C[0][1]=M3+M5;
9     C[1][0]=M2+M4;
10    C[1][1]=M1-M2+M3+M6;
11 }

```

```

1 // Snippet 4
2 void func4(int A[2][2], int B[2][2], int C[2][2]) {
3     int x=A[0][0], y=A[0][1], z=A[1][0], w=A[1][1];
4     x = B[0][0], y=B[0][1], z=B[1][0], w=B[1][1];
5     int M1=x*x, M2=y*z, M3=w*w, M4=y*(x+w), M5=z*(x+w);
6     C[0][0]=M1 + M2;
7     C[0][1]=M4;
8     C[1][0]=M5;
9     C[1][1]=M2 + M3;
10 }

```

- A. Snippet 1 and Snippet 2 will not produce the correct resultant matrix.
 - B. Snippet 1 and Snippet 3 will not produce the correct resultant matrix.
 - C. Snippet 1 and Snippet 4 will not produce the correct resultant matrix.
 - D. Snippet 2 and Snippet 3 will not produce the correct resultant matrix.
 - E. Snippet 2 and Snippet 4 will not produce the correct resultant matrix.
 - F. Snippet 3 and Snippet 4 will not produce the correct resultant matrix.
 - G. All four snippets will not produce the correct resultant matrix.
 - H. The outputs of the snippets may not be the same and changes depending on the input matrix.
 - I. All of the mentioned options are correct.
 - J. None of the mentioned options are correct.
- L
2. (2 points) The following are two code snippets that perform operations on a 2-dimensional array of size $n \times n$, which may contain negative entries and all diagonal entries are always zero. Examine the snippets carefully and choose the option below that best describes their behavior in terms of producing results for all valid inputs. Assume that the same inputs are given to both snippets.

```

1 // Snippet 1
2 #include <stdio.h>
3 #include <limits.h>
4 void compute1(int n, int A[n][n], int src, int result[n]) {
5     int visited[n];
6     for(int i=0;i<n;i++) { result[i]=INT_MAX; visited[i]=0; }
7     result[src]=0;
8     for(int count=0; count<n-1; count++) {
9         int u=-1;
10        for(int i=0;i<n;i++) if(!visited[i] && (u==-1 || result[i]<result[u])) u=i;
11        visited[u]=1;
12        for(int v=0; v<n; v++)
13            if(!visited[v] && A[u][v]!=0 && result[u]+A[u][v]<result[v])
14                result[v]=result[u]+A[u][v];
15    }
16 }
17 int main() {
18     int n=2, src=0;
19     int A[2][2]={0,2},{-1,0};
20     int result[2];
21     compute1(n,A,src,result);
22     for(int i=0;i<n;i++) printf("%d ", result[i]);
23     return 0;
24 }

```

```

1 // Snippet 2
2 #include <stdio.h>
3 #include <limits.h>
4 void compute2(int n, int A[n][n], int src, int result[n]) {
5     for(int i=0;i<n;i++) result[i]=INT_MAX;

```

```

6   result[src]=0;
7   for(int k=0;k<n-1;k++)
8       for(int i=0;i<n;i++)
9           for(int j=0;j<n;j++)
10              if(A[i][j]!=0 && result[i]!=INT_MAX && result[i]+A[i][j]<result[j])
11                  result[j]=result[i]+A[i][j];
12 }
13 int main() {
14     int n=2, src=0;
15     int A[2][2]={0,2},{-1,0};
16     int result[2];
17     compute2(n,A,src,result);
18     for(int i=0;i<n;i++) printf("%d ", result[i]);
19     return 0;
20 }

```

- The contents of the `result` array in Snippet 1 are the same as those in Snippet 2 for the given inputs.
 - The contents of the `result` array in Snippet 1 are the same as those in Snippet 2, if all negative entries in the input arrays of both the snippets are made positive (i.e -2 will be changed to 2).
 - The contents of the `result` array in Snippet 1 are always different from those in Snippet 2.
 - The contents of the `result` arrays in both the snippets will always differ, irrespective of any array inputs.
 - The contents of the `result` array in Snippet 1 may be identical to those in Snippet 2 only for all even-sized arrays.
 - The contents of the `result` array in Snippet 1 may be identical to those in Snippet 2 only for all odd-sized arrays.
 - The contents of the `result` array in Snippet 2 can never be zero for the given input.
 - The contents of the `result` array in Snippet 1 is can never be negative for the given input.
 - The contents of the `result` array in Snippet 2 is can never be negative for any input.
 - All of the mentioned options are correct.
 - None of the mentioned options are correct.
- A, B (1 Mark each)
3. (3 points) The following code snippet processes an integer array of size $n + 1$. Examine it carefully and choose the best option(s) that support the behavior of the code.

```

1 void process(int n, int arr[]) {
2     for(int i=0;i<=n;i++) arr[i]=1;
3     arr[0]=arr[1]=0;
4     for(int i=2;i*i<=n;i++) {
5         if(arr[i]) {
6             for(int j=i*i;j<=n;j+=i)
7                 arr[j]=0;
8         }
9     }
10 }

```

- Indices that correspond to prime numbers up to n remain marked with 1.
 - Every composite index up to n is eventually marked with 0.
 - The array correctly distinguishes prime and composite indices up to n .
 - All even indices greater than 2 remain 1.
 - All odd indices greater than 2 remain 1.
 - Prime indices larger than $\sqrt{n}$ may be incorrectly reset to 0.
 - Composite indices less than n may remain marked as 1.
 - The output pattern of 1's and 0's alternates strictly across indices.
 - The marking process depends on whether n itself is prime or composite.
 - The code produces identical results for all n of the same parity.
 - All of the mentioned options are correct.
 - None of the mentioned options are correct.
- A, B, C (1 Mark each)
4. (2 points) The following code snippet processes an integer array of size n . Examine it carefully and choose the correct option(s) below.

```

1 void process(int arr[], int n) {
2     int max_val = arr[0];
3     for(int i = 1; i < n; i++)
4         if(arr[i] > max_val) max_val = arr[i];
5     int bucket[max_val + 1];

```

```

6   for(int i = 0; i <= max_val; i++) bucket[i] = 0;
7   for(int i = 0; i < n; i++) bucket[arr[i]]++;
8   int index = 0;
9   for(int i = 0; i <= max_val; i++)
10      while(bucket[i]-- > 0) arr[index++] = i;
11 }

```

- A. The function always overwrites the first element with zero.
- B. The function always leaves the last element of the array unchanged.
- C. The function assigns negative values to certain indices.
- D. The function requires n to be even in order to execute.
- E. The function produces identical arrays for all possible inputs.
- F. The function ignores the largest element in the input.
- G. The function doubles every element in the array.
- H. The function shifts every element by one position to the right.
- I. The function erases all elements except the first and last.
- J. The function ensures that the middle element of the array never changes.
- K. All of the mentioned options are correct.
- L. None of the mentioned options are correct.

L

3 Code correction & Numerical Answer Type Questions

Note 1: First, fix all the syntax errors in the code (if any). Then write the single integer answer in the space provided.

Note 2: While correcting the code, circle the wrong /missing syntax and write the correct syntax at the same place. Marks will be given only if all errors are corrected.

[15 points]

1. (3 points) Consider the following C program:

```

1  #include <stdio.h>
2  #include <limits.h>
3  #define MAX 10
4  int arr[MAX][MAX];
5  int s[MAX][MAX];
6  int compute(int dims[], int i, int j) {
7      if (i == j)
8          return 0;
9      if (arr[i][j] != -1)
10         return arr[i][j]
11     int minVal = INT_MAX;
12     int split = -1;
13     for (int k = i; k < j; k++) {
14         int cost = compute(dims, i, k)+ compute(dims, k+1, j)+ dims[i-1] * dims[k] * dims[j];
15         if (cost < minVal) {
16             minVal = cost;
17             split = k;
18         }
19     }
20     arr[i][j] = minVal;
21     s[i][j] == split;
22     return minVal;
23 }
24 int main() {
25     int dims[] = {1; 2, 3, 4};
26     int n = sizeof(dims)/sizeof(dims[0]);
27     for (int i = 0; i < MAX; i++)
28         for (int j = 0; j < MAX; j++)
29             arr[i][j] = s[i][j] = -1;
30     compute(dims, 1, n-1);
31     int sumArr = 0;
32     int sumS = 0;
33     for (int i = 1; i < n; i++) {
34         for (int j = i+1; j < n; j++) {
35             if (arr[i][j] != -1)
36                 sumArr += arr[i][j];
37             if (s[i][j] != -1)
38                 sumS += s[i][j];

```

```

39     }
40 }
41 printf("%d\n", sumArr+sumS);
42 return 0;
43 }

```

What is the output of this program? Write your answer as a single integer.

Line 10 :- arr[i][j];

Line 25 :- int dims[] = 1,2,3,4

Both errors identified 1 Mark or else 0

Output :- 48 or 53 (2 Marks)

2. (3 points) Consider the following C program:

```

1 #include <stdio.h>
2 int iith_d(int x) {
3     return x + 10;
4 }
5 int iith_c(int x) {
6     return iith_d(x - 2);
7 }
8 int iith_b(int x) {
9     return iith_c(x / 2);
10 }
11 int iith_a(int x) {
12     return iith_b(x * 2);
13 }
14 int main() {
15     int distance = 604, x;
16     for (x = 1; x < 3; x++)
17         distance += iith_a(iith_c(x));
18     printf("%d\n", distance);
19     return 0;
20 }

```

What is the output of this program? Write your answer as a single integer.

Line 16 :- for

Line 18 :- ,

Both errors identified 1 Mark or else 0

Output :- 639 (2 Marks)

3. (3 points) Consider the following C program:

```

1 #include <stdio.h>
2 #define MAX 20
3 int storage[MAX], front=0, back=0;
4 void addEnd(int x){
5     storage[back++]=x;
6 }
7 int removeEnd(){
8     return back>0?storage[--back]:0;
9 }
10 void addFront(int x){
11     for(int i=back;i>front;i--)
12         storage[i]=storage[i-1];
13     storage[front]=x;back++;
14 }
15 int removeFront(){
16     if(back==front) return 0;
17     int val=storage[front];
18     for(int i=front;i<back-1;i++)
19         storage[i]=storage[i+1];
20     back--;
21     return val;
22 }
23 int main(){
24     int total=0;
25     addEnd(5); addEnd(3); addEnd(7); addFront(2); addEnd(4); addFront(6); addEnd(1); addEnd(8);
26     total+=removeEnd(); total+=removeFront(); total+=removeEnd();total+=removeFront(); total+=
    removeEnd();
27     printf("%d\n", total);
28     return 0;
29 }

```


What is the output of this program? Write your answer as a single integer.

No Errors (1 Mark)

Output :- 21 (2 Marks)

4. (3 points) Consider the following C program:

```
1 #include <stdio.h>
2 #define SIZE 13
3 int arr[SIZE];
4 int main() {
5     for (int i = 0; i < SIZE; i++) arr[i] = -1;
6     int data[] = {81, 42, 56, 95, 33, 70, 44, 108};
7     int n = sizeof(data)/sizeof(data[0]);
8     for (int i = 0; i < n; i++) {
9         int idx = (data[i] * 7 + 3) % SIZE;
10        while (arr[idx] != -1) {
11            idx = (idx + 3) % SIZE; /* step of three */
12        }
13        arr[idx] = data[i];
14    }
15    long long sum = 0;
16    for (int i = 0; i < SIZE; i++)
17        if (arr[i] != -1)
18            sum += (long long)(i + 1) * (arr[i]);
19    printf("%lld\n", sum);
20    return 0;
21 }
```

What is the output of this program? Write your answer as a single integer.

No Errors (1 Mark)

Output :- 3862 (2 Marks)

5. (3 points) Consider the following C program:

```
1 #include <stdio.h>
2 #include <string.h>
3 /*
4  ASCII values reference:
5  A=65 B=66 C=67 D=68 E=69 F=70 G=71 H=72 I=73 J=74 K=75 L=76 M=77
6  N=78 O=79 P=80 Q=81 R=82 S=83 T=84 U=85 V=86 W=87 X=88 Y=89 Z=90
7  a=97 b=98 c=99 d=100 e=101 f=102 g=103 h=104 i=105 j=106 k=107 l=108 m=109
8  n=110 o=111 p=112 q=113 r=114 s=115 t=116 u=117 v=118 w=119 x=120 y=121 z=122
9  */
10 int main() {
11     char str[] = "AbCdEfGh";
12     int sum = 0;
13     int len = strlen(str);
14     for (int i = 0; i < len; i++) {
15         if (i % 2 == 0)
16             str[i] = str[i] + (i + 3) % 5;
17         else
18             str[i] = str[i] - (i % 4);
19         sum += str[i];
20     }
21     for (int i = 0; i < len/2; i++) {
22         char temp = str[i];
23         str[i] = str[len-1-i];
24         str[len-1-i] = temp;
25     }
26     for (int i = 0; i < len; i++) {
27         if (str[i] >= 'A' && str[i] <= 'Z')
28             sum += str[i] - 'A';
29         else if (str[i] >= 'a' && str[i] <= 'z')
30             sum += str[i] - 'a';
31     }
32     printf("%d\n", sum);
33     return 0;
34 }
```

What is the output of this program? Write your answer as a single integer.

No Errors (1 Mark)

Output :- 706 (2 Marks)

4 Short Answer Questions

Note: Write the answer in the given space provided. Try to write the answer as short as possible. No partial marking.

[26 points]

1. (6 points) At IIT Hyderabad, the student cultural board is preparing the much-awaited annual magazine for Elan & ηVision. Dozens of contributors have submitted their articles, but the real challenge lies in arranging the text neatly on the printed pages. Each submission is broken into a sequence of words. The size of the i -th word, measured in fixed-width characters, is denoted as $w_1, w_2, \dots, w_n$. The editorial team has imposed a strict rule: every line of the magazine can hold at most M characters. The text must be left-aligned, and no word may be split across lines. If a line accommodates words from index p through q (inclusive), then the number of unused spaces at the end of this line is $g = M - \sum_{r=p}^q w_r - (q - p)$. The designers penalize every line by the square of its trailing spaces, i.e., g^2 . The committee's objective is to arrange all words into lines so that the total penalty is minimized. Complete the following C program by filling the blanks (labelled 'BLANK1' ... 'BLANK5') to make up the above objective.

```
1 #include <stdio.h>
2 #define INF 1000000000
3 int min(int a, int b) {
4     return (a < b) ? a : b;
5 }
6 int solve(int w[], int n, int M, int i) {
7     if (i == n) {
8         return __BLANK1__;
9     }
10    int ans = INF;
11    int currLen = 0;
12    for (int j = i; j < n; j++) {
13        currLen += w[j];
14        if (__BLANK2__){
15            break;
16        }
17        int spaceLeft = M - currLen - (j - i);
18        int cost = spaceLeft * spaceLeft;
19        if (j == n - 1){
20            cost = __BLANK3__;
21        }
22        ans = min(ans, __BLANK4__+__BLANK5__);
23    }
24    return ans;
25 }
26 int main() {
27     int n, M;
28     scanf("%d %d", &n, &M);
29     int w[n];
30     for (int i = 0; i < n; i++){
31         scanf("%d", &w[i]);
32     }
33     printf("%d\n", solve(w, n, M, 0));
34     return 0;
35 }
```

(a) (1 point) BLANK1

0

(b) (1 point) BLANK2

currLen+j-i>=M

(c) (1 point) BLANK3

0

(d) (2 points) BLANK4

cost

(e) (1 point) BLANK5

solve(w,n,M,j+1)

Note : Blank4 and Blank5 can be written interchangeably so it will also fetch you mark if you had written the answer for Blank 4 to Blank 5 and Blank5 to Blank 4

2. (10 points) Consider the following C snippet:-

```
1 #include <stdio.h>
2 int main() {
3     char binary[100];
4     int state = 0;
5     int i = 0;
6     printf("Enter a string(containing only digits 0 - 9): ");
7     scanf("%s", binary);
8     while (binary[i] != '\0') {
9         char bit = binary[i];
10        switch (state) {
11            case 0: state = (bit == '0') ? 0 : 1; break;
12            case 1: state = (bit == '0') ? 2 : 3; break;
13            case 2: state = (bit == '0') ? 4 : 5; break;
14            case 3: state = (bit == '0') ? 6 : 0; break;
15            case 4: state = (bit == '0') ? 1 : 2; break;
16            case 5: state = (bit == '0') ? 3 : 4; break;
17            case 6: state = (bit == '0') ? 5 : 6; break;
18        }
19        i++;
20    }
21    if (state == 0)
22        printf("True.\n");
23    else
24        printf("False.\n");
25
26    return 0;
27 }
```

Answer the questions given below using the above C snippet.

- (a) (2 points) If the user enters 111000 as input, what will the program print? What is the state value after processing the second bit of the input string 111000?
program prints True (1 Mark)
state value is 3 (1 Mark)
- (b) (1 point) If the user enters 0, what will the program print?.
program prints True
- (c) (1 point) if a user enters 1012. What output will the program produce and why?
program outputs false (0.5 Mark)
because state value is 4 (0.5 Mark)
- (d) (2 points) Suppose a user pastes a binary string of 200 characters at the prompt. What problems can arise? Will the program necessarily produce the correct result? Explain clearly.
If the user enters a 200-character binary string, the program may exhibit undefined behavior because the array binary is only of size 100. This leads to array overflow, possibly corrupting memory or crashing the program, so the output cannot be trusted. Even if overflow is avoided (say the array is made large enough), the DFA logic will work correctly for arbitrary-length binary inputs, but it may behave unpredictably if digits other than '0' or '1' are provided.
Note:- While correcting you will be given 1 Mark if the word overflow is written. 1 Mark for any proper justification.
- (e) (2 points) Find all the binary strings of length 4, that ends with the program's internal state = 6. Write the string (when converted to decimal value having highest value) and list the state transitions after each bit to show it ends in state 6.
0110, 1101 (1Mark)
For 1101: state0 → state1 → state3 → state6 → state6 (1 Mark)
- (f) (2 points) What is the logic behind the above code snippet? (Hint:- Only consider binary string)
It finds if the given string is in binary format, it verifies whether string is divisible by 7 or not
3. (4 points) You are an IIT-Hyderabad student and it's the end of the semester. You suddenly realize that you have N assignments due immediately, and none of them have been started, so all of them will be late. Each assignment i requires d_i days to complete, and for each day it is late, it incurs a penalty of c_i points. If assignment i ends up being finished t days late, then its penalty is $t \cdot c_i$. Once you start an assignment, you must work on it continuously until it is finished, and you cannot work on multiple assignments at the same time.

For example, suppose you have three assignments. CY1017: Environmental Chemistry takes 3 days and has a penalty of 12 points/day, MA1110: Calculus-I takes 4 days and has a penalty of 20 points/day, and ID1063: ITPL takes 2 days and has a penalty of 4 points/day. The optimal sequence to minimize the total penalty is to do MA1110 first, then CY1017, and finally ID1063. This ordering results in a total penalty of

$$20 \cdot 4 + 12 \cdot (4 + 3) + 4 \cdot (3 + 4 + 2) = 200 \text{ points.}$$

Complete the following C program skeleton by filling in the blanks ('BLANK1', 'BLANK2', 'BLANK3') to implement a strategy that minimizes the total penalty.

```
1 #include <stdio.h>
2 int main() {
3     int N;
4     scanf("%d", &N);
5     int duration[1000], penalty[1000], order[1000];
6     for(int i = 0; i < N; i++) {
7         scanf("%d %d", &duration[i], &penalty[i]);
8         order[i] = i;
9     }
10    for(int i = 0; i < N-1; i++) {
11        for(int j = i+1; j < N; j++) {
12            if(__BLANK1__) {
13                int temp = order[i];
14                order[i] = order[j];
15                order[j] = temp;
16            }
17        }
18    }
19    int currentTime = 0;
20    int totalPenalty = 0;
21    for(int k = 0; k < N; k++) {
22        int i = order[k];
23        __BLANK2__;
24        currentTime += __BLANK3__;
25    }
26    printf("%d\n", totalPenalty);
27    return 0;
28 }
```

- (a) (2 points) BLANK1

$$\frac{\text{(double) penalty[order[i]]}}{\text{duration[order[i]]}} < \frac{\text{(double) penalty[order[j]]}}{\text{duration[order[j]]}}$$
- (b) (1 point) BLANK2

$$\text{totalPenalty} += \text{penalty[i]} \times \text{currentTime};$$
- (c) (1 point) BLANK3

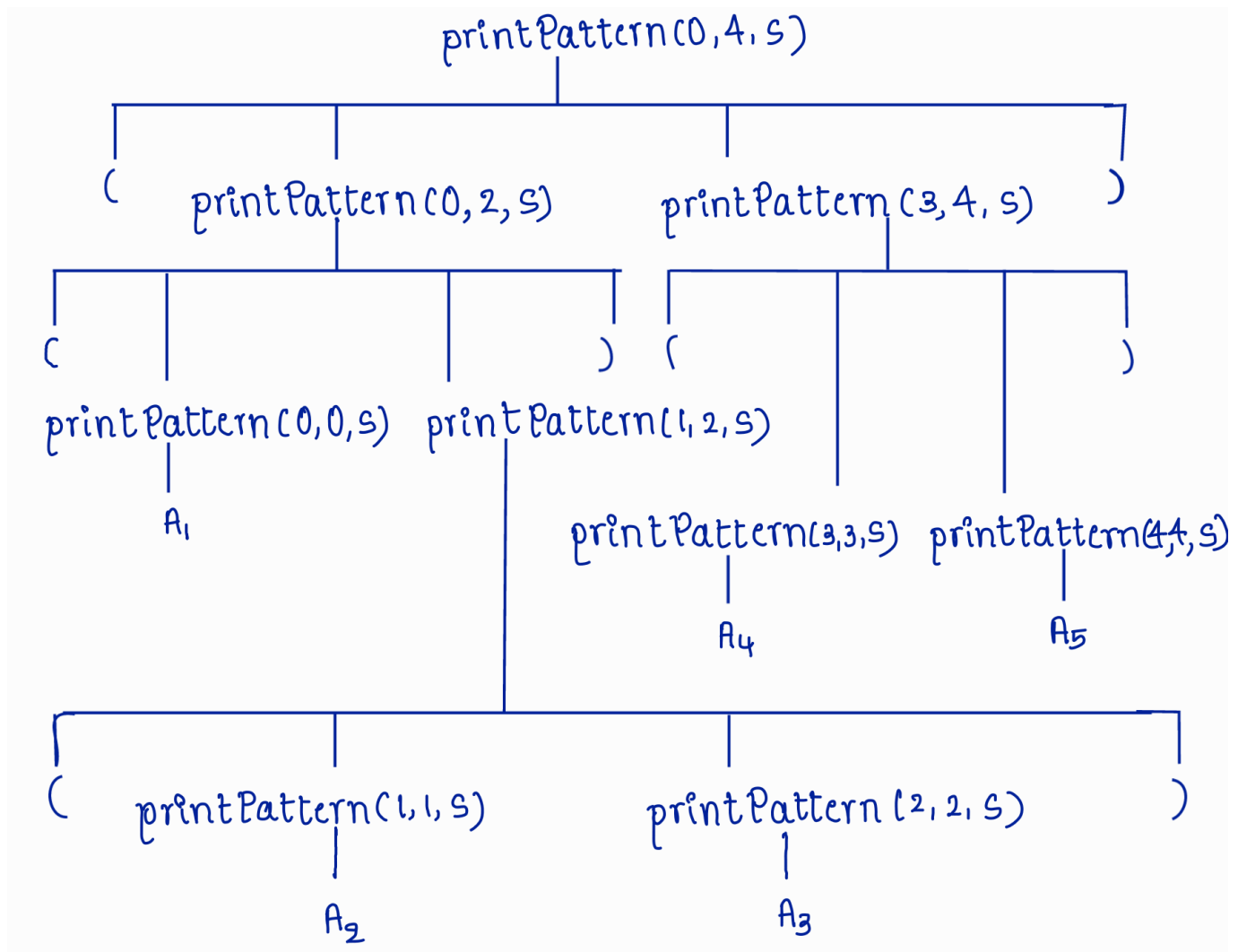
$$\text{duration[i]}$$

4. (6 points) Consider the following C snippet:

```
1 #include <stdio.h>
2 #define N 5
3 void printPattern(int i, int j, int s[N][N]) {
4     if (i == j) {
5         printf("A%d", i+1);
6         return;
7     }
8     printf("(");
9     printPattern(i, s[i][j]-1, s);
10    printPattern(s[i][j], j, s);
11    printf(")");
12 }
13 int main() {
14     int s[N][N] = {
15         {-1, 1, 1, 3, 3},
16         {-1,-1, 2, 3, 3},
17         {-1,-1,-1, 3, 3},
18         {-1,-1,-1,-1, 4},
19         {-1,-1,-1,-1,-1}
20     };
21     printPattern(0, N-1, s);
22     return 0;
23 }
```

Answer the following questions using the above code snippet.

- (a) (4 points) Draw the recursion calls for the call `printPattern(0,4,s)`. (Show all recursive calls and splits.)
 Note:- No need to draw the outputs just recursion calls are enough Note :- Each level correctly drawn will be



given 1 Mark

- (b) (2 points) What is the output printed on the screen when the above program is executed?
 ((A1(A2A3))(A4A5))

Note :- Marks will be given only if entire output is correct. No partial marking

5 Long Answer Questions

Note: Write the answer in the given space provided. Comment your function properly to clearly specify what your code is doing.

[30 points]

In any code the evaluation goes as follows:-

1. Syntax Perfection - 1Mark
 2. Correctness - 1 Mark
 3. Logic - Will get remaining marks
1. (6 points) Consider the following incomplete C program. It reads a binary string from the user and then calls two functions: one to check whether the string represents a number (when the binary string is converted to decimal number) divisible by 5 and another to check whether it is divisible by 4 .

The functions must be implemented using only 'switch' statements. You are not allowed to convert the binary string to decimal or use built-in conversion functions.

```
1 #include <stdio.h>
2 #include <string.h>
3 // Function prototypes (to be implemented by you)
4 int isDivisibleBy5(char str[]);
5 int isDivisibleBy4(char str[]);
6 // Assume main function is already implemented and it will print true, if the return value of the
   function is 1 and prints false if the return value of the function is 0.
```

- (a) (3 points) Implement the function `int isDivisibleBy5(char str[])` so that it checks divisibility by 5 using only a switch statement. Write the complete function definition.

Solution

```
1 int isDivisibleBy5(char str[]) {
2     int state = 0;
3     int i = 0;
4     while (str[i] != '\0') {
5         char bit = str[i];
6         switch (state) {
7             case 0:
8                 switch (bit) {
9                     case '0': state = 0; break;
10                    case '1': state = 1; break;
11                }
12                break;
13            case 1:
14                switch (bit) {
15                    case '0': state = 2; break;
16                    case '1': state = 3; break;
17                }
18                break;
19            case 2:
20                switch (bit) {
21                    case '0': state = 4; break;
22                    case '1': state = 0; break;
23                }
24                break;
25            case 3:
26                switch (bit) {
27                    case '0': state = 1; break;
28                    case '1': state = 2; break;
29                }
30                break;
31            case 4:
32                switch (bit) {
33                    case '0': state = 3; break;
34                    case '1': state = 4; break;
35                }
36                break;
37        }
38        i++;
39    }
40    switch (state) {
41        case 0: return 1; // divisible by 5
42        default: return 0; // not divisible
43    }
44 }
```

- (b) (3 points) Implement the function `int isDivisibleBy4(char str[])` so that it checks divisibility by 4 using only a switch statement. Again, do not convert the string to decimal; use switch logic only. Write the complete function definition.

Solution

```
1 int isDivisibleBy4(char str[]) {
2     int len = strlen(str);
3     char b1 = '0', b2 = '0';
4     int state = 0;
5
6     switch (len) {
7         case 0: return 0; // empty string
8         case 1:
```

```

9         b2 = str[len - 1];
10        break;
11    default:
12        b1 = str[len - 2];
13        b2 = str[len - 1];
14        break;
15    }
16
17    switch (b1) {
18        case '0':
19            switch (b2) {
20                case '0': state = 0; break;
21                case '1': state = 1; break;
22            }
23            break;
24        case '1':
25            switch (b2) {
26                case '0': state = 2; break;
27                case '1': state = 3; break;
28            }
29            break;
30    }
31
32    switch (state) {
33        case 0: return 1; // divisible by 4
34        default: return 0; // not divisible
35    }
36 }

```

2. (8 points) You are given three integer arrays A, B, and C, each of size n. You need to determine whether there exist elements $a \in A$, $b \in B$, and $c \in C$ such that:

$$a + b + c = 0$$

- (a) (4 points) Complete the following C program. It reads three arrays of size n from the user and then calls hasZeroSum().

```

1 #include <stdio.h>
2 // Function prototypes ( to be implemented by you )
3 int hasZeroSum(int A[], int B[], int C[], int n);
4 // Assume main function is already implemented and it will print true, if the return value of
   the function is 1 and prints false if the return value of the function is 0.

```

Solution

```

1 // Function to check if there exist a, b, c such that a + b + c = 0
2 int hasZeroSum(int A[], int B[], int C[], int n) {
3     for (int i = 0; i < n; i++) {
4         for (int j = 0; j < n; j++) {
5             for (int k = 0; k < n; k++) {
6                 if (A[i] + B[j] + C[k] == 0)
7                     return 1; // Found a valid triplet
8             }
9         }
10    }
11    return 0; // No such combination found
12 }

```

- (b) (4 points) The program below reads three integer arrays A, B, and C (each of length n). It also computes and stores every sum $a + b$ for $a \in A$, $b \in B$ into the array table[].

Complete the function `int hasZeroSumUsingTable(int C[], int nC, int table[], int tSize)` so that it uses the precomputed values in `table[]` and the elements of `C` to determine whether there exists a combination of elements from the three arrays that satisfies the required condition. Return 1 if such a combination exists, otherwise return 0.

Do not use any library search routines — write all logic explicitly in C.

```

1 #include <stdio.h>
2 #define MAX 100
3
4 // Function prototype (to be implemented by you)
5 int hasZeroSumUsingTable(int C[], int nC, int table[], int tSize);

```

```

6
7 int main() {
8     int n, A[MAX], B[MAX], C[MAX];
9     printf("Enter n: ");
10    scanf("%d", &n);
11
12    printf("Enter elements of A: ");
13    for (int i=0; i<n; i++) scanf("%d", &A[i]);
14
15    printf("Enter elements of B: ");
16    for (int i=0; i<n; i++) scanf("%d", &B[i]);
17
18    printf("Enter elements of C: ");
19    for (int i=0; i<n; i++) scanf("%d", &C[i]);
20
21    // Build table of all a+b values
22    int table[MAX*MAX];
23    int tSize = 0;
24    for (int i=0; i<n; i++) {
25        for (int j=0; j<n; j++) {
26            table[tSize++] = A[i] + B[j];
27        }
28    }
29
30    if (hasZeroSumUsingTable(C,n,table,tSize))
31        printf("There exists a triplet with zero sum.\n");
32    else
33        printf("No such triplet exists.\n");
34
35    return 0;
36 }
37
38 // Implement this function to use table[] and elements of C
39 int hasZeroSumUsingTable(int C[], int nC, int table[], int tSize) {
40     // Write your code here
41 }

```

Solution

```

1 int hasZeroSumUsingTable(int C[], int nC, int table[], int tSize) {
2     // For each element in C, check if its negation exists in table[]
3     for (int i = 0; i < nC; i++) {
4         int target = -C[i];
5         for (int j = 0; j < tSize; j++) {
6             if (table[j] == target)
7                 return 1; // Found (a + b) = -c, hence a + b + c = 0
8         }
9     }
10    return 0; // No valid triplet found
11 }

```

3. (6 points) In a digital signal-processing experiment at IITH, a sensor produces a sequence of positive integer readings `readings[]` of size `n`. A sequence is said to be **balanced** if it is possible to choose some subset of these readings, multiply each chosen reading by any integer (positive, negative, or zero), and combine them so that the total equals exactly 1.

Write a C program that reads the readings and determines whether the sequence is balanced. Implement a recursive function `int isBalanced(int readings[], int n)` that returns 1 if the sequence is balanced and 0 otherwise.

```

1 #include <stdio.h>
2 // Function prototype
3 int isBalanced(int readings[], int n);
4 // Assume main function is already implemented and it will print balanced, if the return value of
   the function is 1 and prints not balanced if the return value of the function is 0.

```

Sample Input/Output:

- **Input:** readings = [8, 11, 17]

Output: Balanced

Explanation: It is possible to find integers x, y, z such that $x \times 8 + y \times 11 + z \times 17 = 1$. (For example, $-3 \times 11 + 17 \times (2) = 1$.)

- **Input:** readings = [14, 25, 9, 31]
Output: Balanced
Explanation: Integers x_i exist so that $x_1 \times 14 + x_2 \times 25 + x_3 \times 9 + x_4 \times 31 = 1$.
- **Input:** readings = [4, 6]
Output: Not balanced
Explanation: No choice of integers x, y satisfies $x \times 4 + y \times 6 = 1$.

Solution

```

1 #include <stdio.h>
2 // Function to compute gcd of two numbers
3 int gcd(int a, int b) {
4     if (b == 0)
5         return a;
6     return gcd(b, a % b);
7 }
8 // Recursive function to check if sequence is balanced
9 int isBalanced(int readings[], int n) {
10     if (n == 1)
11         return (readings[0] == 1) ? 1 : 0;
12     int currentGCD = readings[0];
13     for (int i = 1; i < n; i++)
14         currentGCD = gcd(currentGCD, readings[i]);
15     return (currentGCD == 1) ? 1 : 0;
16 }

```

4. (6 points) In a campus tour planning experiment at IITH, a route consists of $2n+1$ landmarks, each with a popularity rating represented as a real number. The ratings are given in an unsorted array `landmarkRatings[]` of size $2n+1$. A route is said to be zigzag if the popularity ratings along the route alternate in the following pattern:

$$R[1] \leq R[2] \geq R[3] \leq R[4] \geq \dots \leq R[2n] \geq R[2n+1]$$

Write a C program that reads the ratings and produces a permutation of the landmarks such that the resulting route is zigzag. Implement a function `void arrangeZigzag(double landmarkRatings[], int n)` that rearranges the array to satisfy the zigzag property.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 // Function prototypes ( to be implemented by you )
4 void arrangeZigzag(double landmarkRatings[], int n);
5 // Assume that main function is already implemented and it calls the function arrangeZigzag using
   pass by reference strategy.

```

Solution

```

1 void arrangeZigzag(double landmarkRatings[], int n) {
2     int size = 2 * n + 1;
3     // Sort the array first
4     for (int i = 0; i < size - 1; i++) {
5         for (int j = 0; j < size - i - 1; j++) {
6             if (landmarkRatings[j] > landmarkRatings[j + 1]) {
7                 double temp = landmarkRatings[j];
8                 landmarkRatings[j] = landmarkRatings[j + 1];
9                 landmarkRatings[j + 1] = temp;
10            }
11        }
12    }
13    // Swap adjacent elements to create zigzag pattern
14    for (int i = 1; i < size - 1; i += 2) {
15        double temp = landmarkRatings[i];
16        landmarkRatings[i] = landmarkRatings[i + 1];
17        landmarkRatings[i + 1] = temp;
18    }
19 }

```

5. (4 points) The IIT Hyderabad Astronomy Club wants to estimate the value of Euler's number e using the series expansion up to the n -th term:

$$e = \sum_{k=0}^n \frac{1}{k!}$$

Write a C program that computes e using this formula for $n = 20$. Implement a function `double computeE(int n)` that returns the computed value of e .

```
1 #include <stdio.h>
2 // Function prototypes (to be implemented by you)
3 double computeE(int n);
4 //Assume that main function is already implemented and calls the function computeE using pass by
   value strategy.
```

Solution

```
1 // Function to compute e using series expansion
2 double computeE(int n) {
3     double e = 1.0;      // first term 1/0! = 1
4     double fact = 1.0;    // to store factorial values
5     for (int k = 1; k <= n; k++) {
6         fact *= k;        // compute k!
7         e += 1.0 / fact;  // add 1/k! to sum
8     }
9     return e;
10 }
```